2024 IEEE 30th International Symposium for Design and Technology in Electronic Packaging (SIITME) | 979-8-3315-3951-1/24/$31.00 ©2024 IEEE | DOI: 10.1109/SIITME63973.2024.10814877

A Reference Architecture for Deploying Large Language Model Applications in Industrial Environments

Felix Mahr
Friedrich-Alexander-Universität Erlangen-Nürnberg (FAU), Institute for Factory Automation and Production Systems (FAPS)
Nuremberg, Germany
Felix.Mahr@faps.fau.de

Giulia Angeli
Siemens AG
Amberg, Germany

Till Sindel
Friedrich-Alexander-Universität Erlangen-Nürnberg (FAU), Institute for Factory Automation and Production Systems (FAPS)
Erlangen, Germany
0009-0004-3507-631X

Konstantin Schmidt
Siemens AG
Amberg, Germany
0000-0002-4857-4121

Jörg Franke
Friedrich-Alexander-Universität Erlangen-Nürnberg (FAU), Institute for Factory Automation and Production Systems (FAPS)
Erlangen, Germany
0000-0003-0700-2028

***Abstract*—Integrating Large Language Models (LLMs) into industrial environments presents numerous challenges. Firstly, understanding the term LLM, its required components for implementation, and the various types of LLMs demands research effort. Furthermore, the processing of textual data involves enormous data volumes, and executing large language models requires substantial computational power. A significant challenge in model deployment on the shopfloor is the stringent data security measures, which often prohibit direct internet access to the cloud.**

To address these challenges, a reference architecture was developed to give an overview of the components for integrating LLM applications in industry. This architecture covers all essential steps of LLM applications, including data preparation, model training, model evaluation, model deployment, and prompt engineering. For model deployment, two possible locations were distinguished: the digital workplace, such as the office, and the shopfloor. In addition to model deployment, data preparation and model evaluation are emphasized as critical elements for the success of Large Language Model Operations (LLMOps) applications.

This reference architecture enables project teams to better understand the elements involved in the LLM integration process. Furthermore, the universal approach helps in standardizing the associated processes, facilitating a more effective acceptance of LLMs in industrial environments.

Keywords—electronics production, machine learning, large language models, large language model operations

I. Introduction

The characteristic of big companies of having vast knowledge that cannot be accessed are seen as a weakness. Now through the potential of large language models (LLM), the knowledge will be collected and harnessed via this technology. Hence, the initial existing gap is closed, and the knowledge is spread through the development of domain specific LLM applications using internal data.

Compared to the digital workplace, implementing Large Language Model Operations (LLMOps) applications in the shopfloor remains a challenge. While LLM-based solutions are available for digital workplaces, establishing the connection between shopfloor and cloud-hosted LLMs remains a challenge. These connectivity issues are a result of security constraints regarding a bidirectional production to cloud level connectivity. In addition, the limited local computing resources and storage capacities present significant challenges for running LLM applications locally in production due to their high demands on computing power and storage.

This leads to the topic of this research to develop a holistic approach on how to integrate LLMOps applications and depict possible connectivity solutions for use-cases in the production. Within that holistic approach, different model strategies, such as Finetuning or Retrieval Augmented Generation (RAG), of LLM applications, but also implementation strategies are included. This makes decisions visible at different stages of the development process, that are ensuring the success of the outcome. As an initial overview of the integration process of the LLMOps application, the reference architecture includes all necessary steps of data collection and preparation, the development, the evaluation, and the final deployment. Therefore, this research answers the question of what needs to be done to integrate an LLMOps application.

To get this question answered, the goal of this research is the development of a reference architecture for LLMOps applications. With the help of the reference architecture, a project team starting with the development of an LLMOps application gets an overview of the necessary steps for their project. That encourages creating a common understanding of the topic and a clear framework about LLMOps applications.

II. Background of this Investigation

As LLMs are a subcategory of large foundation models created using Machine Learning (ML), it is important to also consider the fundamental principles of Machine Learning Operations (MLOps) for LLMOps applications [1]. For the development of LLMOps applications, the MLOps framework by [2] is adaptable to serve as a basis for the reference architecture.

Authorized licensed use limited to: UNIVERSITAETSBIBL STUTTGART. Downloaded on February 25,2026 at 15:42:53 UTC from IEEE Xplore. Restrictions apply.

In the MLOps framework, the three levels describe an increasing degree of automated processes [2]. As the reference architecture suggested within this work presents the fundamentals of LLMOps applications integration, the first level of the MLOps framework serves as starting point. The basic steps of this MLOps Level 0 are as follows [2]:

1. Data extraction
2. Data analytics
3. Data preparation
4. Model training
5. Model evaluation
6. Model validation
7. Model deployment
8. Model monitoring

Compared to MLOps, LLMOps use-cases consist of different operational steps. These are summarized in the developed reference architecture that can be seen in Figure 1.

To develop a suitable reference architecture, a structured approach helps to identify important elements. Therefore, the methodological approaches of [3], [4], and [5] were utilized to define the proposed steps of the reference architecture.

III. CHALLENGES OF LARGE LANGUAGE MODELS IN MANUFACTURING INDUSTRY

The shopfloor use-cases hold the potential to enhance the effectiveness of operations, the optimization of costs, and strategic benefits on the production plant. However, as there are cybersecurity restrictions for the bidirectional shopfloor-cloud-connectivity and high computing resources required, the need arises to develop a methodology on how this problem can be solved proposing different approaches.

The possibilities for establishing a connection to the cloud are limited to harnessing API calls and edge computing. Due to the complexity of such disruptive applications, a holistic approach is needed.

To receive a holistic approach, the development of a reference architecture is an advantageous methodology [3]. As Natural Language Processing (NLP) remains a new topic in the industrial environment, the complexity needs to be reduced to its key elements of the integration process to ensure the understanding of the topic and the challenges that might occur when implementing the technology.

Further, a reference architecture provides a universal vocabulary for all stakeholders. In large companies, the wording needs to be clearly defined to ensure the correct understanding of discussed topics. Also, if parts of the deployment process of LLMOps applications are specific to this technology, it might help to clarify the stakeholders, what is particularly important for the development. [3]

Additionally, a reference architecture broken down to its key elements is scalable for other use-cases besides the shopfloor. All necessary steps need to be included which are necessary for the integration of LLMOps applications. In conclusion, the reference architecture is not only applied for the initial use-case but also as a general standard. Especially for LLMOps applications, which might be useful on the shopfloor as well as in a digital workplace, a method is necessary that enables seamless access to the application from both locations. This is particularly important in the context of IT/OT convergence, where the integration of information technology and operational technology allows for enhanced efficiency and collaboration between digital and physical environments. [3]

IV. THE REFERENCE ARCHITECTURE FOR LARGE LANGUAGE MODEL OPERATIONS APPLICATIONS

The reference architecture for LLMOps applications is presented in Figure 1, with the legend in the top right corner. From the top to the bottom the five LLM development steps are presented. First, the Data Preparation is conducted before the data is pulled to the Model Strategy in the next step. After the LLM is fed with the data, it needs to be evaluated and validated in the step of Model Evaluation by first quantitative metrics. If the quantitative evaluation is sufficient, the LLM is deployed in the Model Deployment step to its location, of either the shopfloor or the digital workplace. Otherwise, additional measures to achieve the desired performance must be conducted. As an additional element, the Prompt Engineering is included as a perspective for the end user or automated implementation in the backend of the application. Lastly, to ensure that the qualitative evaluation is sufficient, especially for domain-specific use-cases, human feedback is collected for finetuning purposes.

A. Data Preparation

As a first implementation step of the reference architecture, the Data Preparation is located on top. This step describes how the data is converted into the LLMs required format. As LLMs are trained on embeddings which are multidimensional vectors representing textual meanings, this process is executed during this step [6]. Included are the steps from retrieving the original data to the embedding process of these and the final storage of the embeddings.

The Data Preparation starts with the Data Source containing the textual data in their original format. This can be e.g. a documentation in a pdf format, a word document or a text document. The documents are uploaded to a central Knowledge Base, potentially cloud-hosted, where they are further divided into semantically organized Chunks. Subsequently, the text chunks are split into Tokens using a Tokenizer. The Tokens are afterwards converted into the desired data format as Vectors with the help of a Vectorizer.

A Vector Database is used to store the vectorized data. As the data lake is a term for a storage in which all data is saved, it may be cloud-hosted or an on-premises solution. As the reference architecture is supposed to support different kinds of solutions the data lake and database instance are therefore separated. In conclusion, there might be a transfer to the data lake necessary if the internal infrastructure demands it.

B. Model Strategies

The second implementation step involves selecting a Model Strategy, categorized into five approaches: Development from scratch, Commercial Model, Parameter-Efficient Finetuning (PEFT), Full Finetuning, and Retrieval-Augmented Generation (RAG). These strategies were identified through a comprehensive analysis of existing methodologies and have been simplified in the reference architecture to their core components.

Authorized licensed use limited to: UNIVERSITAETSBIBL STUTTGART. Downloaded on February 25,2026 at 15:42:53 UTC from IEEE Xplore. Restrictions apply.

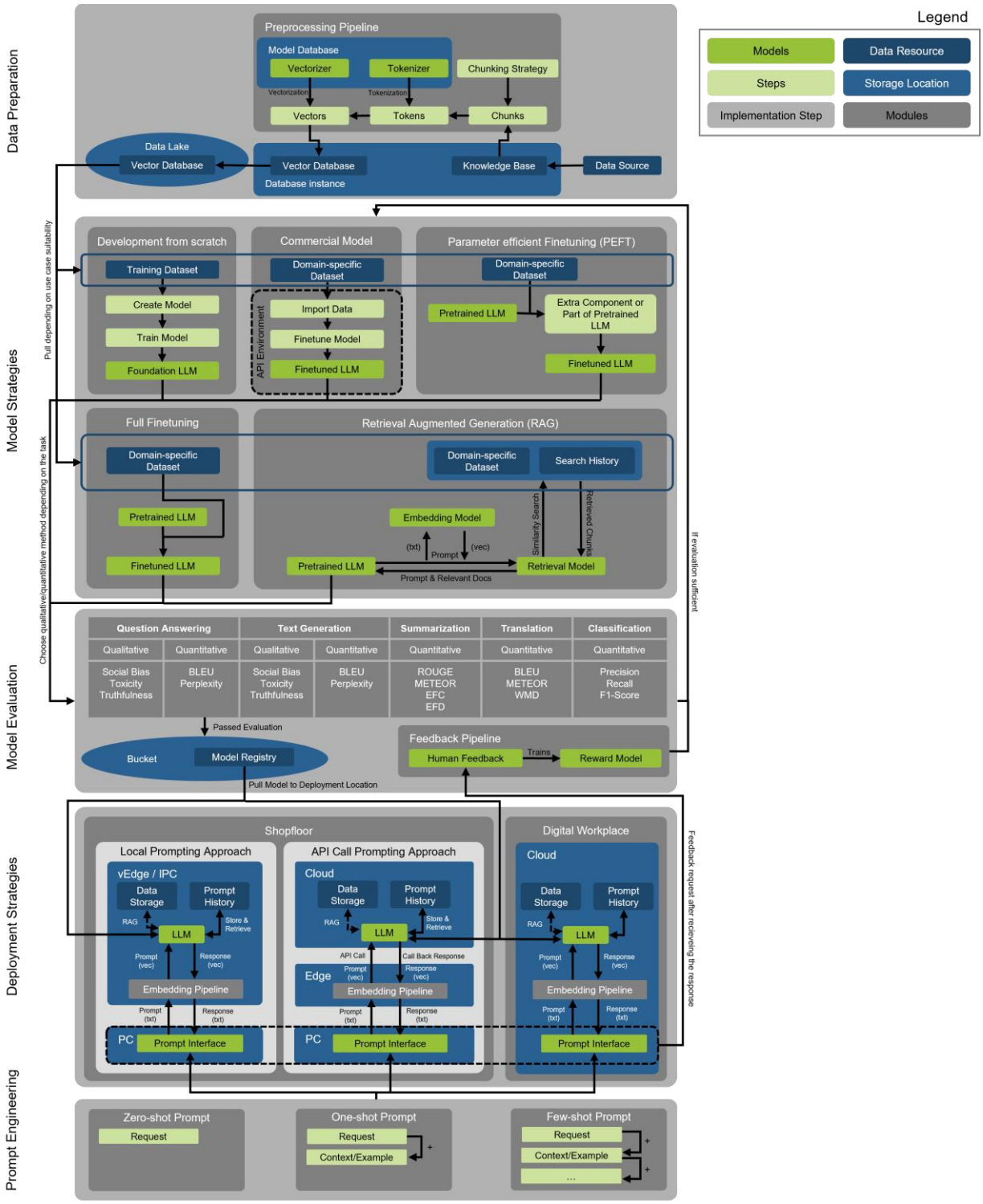

Fig. 1: The Reference Architecture for Deploying Large Language Model Applications in Industrial Environments.

Authorized licensed use limited to: UNIVERSITAETSBIBL STUTTGART. Downloaded on February 25,2026 at 15:42:53 UTC from IEEE Xplore. Restrictions apply.

The first strategy of Development from scratch is the most time and cost-intensive option because of the need to start from zero. As in every other strategy, first, a training or domain-specific dataset needs to be developed which is vectorized. Afterwards, the model is developed which can be done with the common python packages [7]. Afterwards, this model is trained with the training dataset to create the foundation model, which is directly used or further finetuned.

Due to the high number of neurons needed to be trained for language models, which usually includes a training dataset size of billions of words, the internal development option is often not suitable [8]. Therefore, like with OpenAI APIs, a commercial pre-trained model is fine-tuned with a domain-specific dataset that is imported into the provider's environment. With the use of this approach, domain-specific use-cases are conducted as well by harnessing the commercial models, which are performing best across a multitude of scoring metrics.

In comparison to the Commercial Models, the Full Finetuning model strategy does not run in an API environment. But it also uses a pretrained LLM which is fully finetuned on a domain-specific dataset.

A further model strategy is PEFT. In this approach, the domain-specific dataset and the pretrained LLM are integrated into an additional component where only a subset of the pre-trained LLM's parameters is finetuned. This means that only a portion of the pretrained LLM is finetuned, rather than the entire model, which reduces both costs and time compared to the Full Finetuning approach.

The last model strategy is RAG, which works with an additional retrieval model. When a prompt enters the RAG system, it is converted into an embedding, which is further transferred to the retrieval model. The retrieval model conducts a similarity search to retrieve semantically related data points from the domain-specific dataset, that are stored in a vector database. Additionally, this method takes the search history into account. That information is sent back to the LLM to generate the answer based on the retrieved data points and the initial prompt. [9]

C. Model Evaluation

In the following implementation step, the model is evaluated with suitable metrics or methods depending on the desired task the LLM must perform. In this presentation, only a few fundamental metrics were included to give an exemplary overview of possible options.

For the tasks of question answering and text generation, both qualitative and quantitative evaluation methods are included. This is necessary because these tasks generate their outputs based on the input received during the training process. The input prompt of these tasks usually does not include much information that is used for the generated output [10]. For tasks like summarization, translation, and classification the input includes information that the LLM uses to perform the task. Therefore, the qualitative evaluation might not be crucial for other tasks than question answering and text generation.

Quantitative metrics provide numerical values to assess the quality of a model, but their significance increases when the model is validated against a high-performing reference model. By comparing the evaluated model's metrics to those of a proven, high-performance model, the validity and relevance of the metrics is enhanced.

Even though the metrics work best chosen according to the associated tasks, they are also applied on other tasks to provide a comprehensive overview of the LLM. Qualitative evaluation becomes particularly beneficial when human feedback is utilized to train a model via reinforcement techniques, which subsequently is employed for continuously training an existing LLM [11]. This human feedback is part of a qualitative evaluation when experts give feedback on the truthfulness or social bias of a model.

The entire evaluation process is iterative and therefore uses multiple evaluation methods to assess different aspects of the LLMs performance. Once the LLM passed the evaluation, it is stored in a model repository. This helps to keep track of the changes to the model and how it improves over time. From there, the model is pulled to the desired deployment location.

D. Model Deployment

Once the model passes the evaluation, it will be deployed to make it accessible to the end user in the next implementation step. For the deployment, different approaches are considered due to requirements of cybersecurity and limitations of computing resources. Therefore, three deployment strategies are included in the reference architecture.

Firstly, the deployment solutions are differentiated between shopfloor and digital workplace. As discussed, the shopfloor is located in the production, while the digital workplace covers the standard office spaces. As previously mentioned, the shopfloor and digital workplace are especially different in their access to the cloud: While computers in the digital workplace can access the internet, computers in the production are mostly decoupled from the internet due to cybersecurity measures. Therefore, a solution for the shopfloor that can access the cloud indirectly through an API Call Prompting Approach is needed. Alternately, the LLMOps application can run locally with a Local Prompting Approach.

First, a Local Prompting Approach is realized through deploying the LLM on a virtual edge device (vEdge) or industrial PC (IPC) having higher computing resources available. A prompt interface is implemented on the personal computer which represents the interface where the end user enters the prompt. The prompt must then be transformed into a vector representation to enable the LLM to process it [6]. Therefore, the embedding pipeline is interposed which functions similarly to the embedding pipeline described in the data preparation step. In reverse, the response of the LLM is decoded back into a textual answer.

Secondly, as in the Local Prompting Approach, for the API Call Prompting Approach, the prompt interface is located again on the personal computer and the embedding pipeline is located on the edge device. In this solution, the LLM is accessed over an API call as it is stored in the cloud. The 6

Authorized licensed use limited to: UNIVERSITAETSBIBL STUTTGART. Downloaded on February 25,2026 at 15:42:53 UTC from IEEE Xplore. Restrictions apply.

E. Prompt Engineering

To harness the full potential of the LLMOps application, the correct prompting technique can have a huge influence on the generated output and its quality to solve the problem which is aimed to be solved with the help of the LLM. Therefore, prompting techniques are included in the reference architecture as a last implementation step.

Based on the faced problem type, a corresponding prompting technique is chosen. The first technique of Zero-shot Prompting is a simple request that does not require context or further information about the environment. The second technique of One-shot Prompting includes one example or context which helps the LLM to generate a more suitable answer. Lastly, Few-shot Prompting describes a prompt where more than one example or context is given besides the request. [10]

V. CONCLUSION AND OUTLOOK

The goal of this paper was to develop a reference architecture for LLMOps applications which depicts the relevant implementation steps of such systems. With the help of the reference architecture, a project team developing a new LLM use-case quickly gains an overview of all the necessary components in the integration process.

However, the absence of a comprehensive overview of all components for LLM applications highlighted the need for a holistic reference architecture. Therefore, multiple use-cases were considered for the integration of LLMOps applications. The reference architecture consists of the following five components needed in LLM applications: Data Preparation, Model Strategies, Model Evaluation, Deployment Strategies, and Prompt Engineering. To make the reference architecture scalable, five model strategies are incorporated. Further, three deployment strategies are developed differentiating between shopfloor and digital workplaces addressing data security and local computing resource issues on the shopfloor.

The Data Preparation and Model Evaluation steps were identified as particularly challenging. In addition to the complexity of Data Preparation, which involves creating embeddings, storage also poses a significant challenge. Numerous methods are available for both steps, with the appropriate method selected based on the specific use case. However, due to the large number of methods, the evaluation is typically conducted at a surface level.

As the presented reference architecture is the theoretical approach to integrate LLMOps applications in a factory, the practical implementation into a real environment is subject of future research analyzing all its pros and cons for individual use-case types. Given the beneficial impact LLMOps applications can have in industry, LLMs will continue to be a prominent research focus with significant potential for the sustainable development of industrial environments.

REFERENCES

[1] JANIESCH, C., P. ZSCHECH and K. HEINRICH. Machine learning and deep learning [online]. Electronic Markets, 2021, 31(3), pp.685-695. ISSN 1019-6781. Available under: doi:10.1007/s12525-021-00475-2

[2] GOOGLE. MLOps [online]. MLOps: Continuous delivery and automation pipelines in machine learning, 2020 [Access on: 25th April 2024]. Available under: https://cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning

[3] PATTON, J.S. and J.D. MORELAND. Reference Architecture. In: D. VERMA, Hg. Systems Engineering for the Digital Age: Wiley, 2023, pp. 683-695. ISBN 9781394203284

[4] REIDT, A. Referenzarchitektur eines integrierten Informationssystems zur Unterstützung der Instandhaltung. Dissertation. München, 2019

[5] ANGELOV, S., P. GREFEN and D. GREEFHORST. A classification of software reference architectures [online]. Analyzing their success and effectiveness. 2009 Joint Working IEEE/IFIP Conference on Software Architecture & European Conference on Software Architecture, 2009, pp.141-150. Available under: doi:10.1109/WICSA.2009.5290800

[6] ASHISH VASWANI, NOAM SHAZEER, NIKI PARMAR, JAKOB USZKOREIT, LLION JONES, AIDAN N. GOMEZ, ŁUKASZ KAISER and ILLIA POLOSUKHIN. Attention is All you Need. Advances in Neural Information Processing Systems, 2017, 30

[7] WEI, M., N.S. HARZEVILI, Y. HUANG, J. WANG and S. WANG. Demystifying and Detecting Misuses of Deep Learning APIs. In: M.B. DWYER, D. DAMIAN und A. ZELLER, Hg. Proceedings of the 44th International Conference on Software Engineering. New York, NY, USA: ACM, 2022, pp. 376-387. ISBN 9781450392211

[8] CARREIRA, S., T. MARQUES, J. RIBEIRO und C. GRILO. Revolutionizing Mobile Interaction. Enabling a 3 Billion Parameter GPT LLM on Mobile, 29. September 2023

[9] LEWIS, P., E. PEREZ, A. PIKTUS, F. PETRONI, V. KARPUKHIN, N. GOYAL, H. KÜTTLER, M. LEWIS, W. YIH, T. ROCKTÄSCHEL, S. RIEDEL and D. KIELA. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks [online]. NIPS '20: Proceedings of the 34th International Conference on Neural Information Processing Systems, 2020, 34, pp.9459-9474. Available under: doi:10.48550/arXiv.2005.11401

[10] AMAZON. Prompt Engineering guidelines. In: Amazon Bedrock. User Guide, 2024, pp. 190-233

[11] LIU, G.K.-M. Transforming Human Interactions with AI via Reinforcement Learning with Human Feedback (RLHF), 28. Februar 2023

Authorized licensed use limited to: UNIVERSITAETSBIBL STUTTGART. Downloaded on February 25,2026 at 15:42:53 UTC from IEEE Xplore. Restrictions apply.